Practical Problems: Algorithmic Thinking

Learning Outcome Addressed

- Identify whether a problem can be solved by following an algorithm, applying a pattern, or problem-solving.
- Use top-down design to break up medium-sized programming tasks into smaller pieces and solve each piece individually.

This assignment allows you to apply what you have learned in this module about algorithms, patterns, problem-solving, and top-down design. Follow the instructions below to complete problems one to five. If you get stuck on a problem and are not making any progress, please reach out to the learning facilitator for assistance.

1. [20 pts] convert_string_to_number

Write a function **convert_string_to_number(s)** which takes a string containing only alphabetical letters and converts it to a string using the following method: visit each letter in the string, map each letter to a number (1 for a, 2 for b, 3 for c, etc), and add all the mapped numbers together. For example, convert_string_to_number("apple") would return 50 because "a" maps to 1, "p" maps to 16, "l" maps to 12, and "e" maps to 5, and 1+16+16+12+5=50. You should treat uppercase letters the same as lowercase – the built-in method .lower() may be helpful here.

You can compute each number from the corresponding letter easily by making all the letters lowercase and using the built-in ord method to map each character to its ASCII value (ord("a") returns 97, ord("b") returns 98, ord("c") returns 99, etc). Just offset the ASCII value by 96 to get the correct number.

Hint: read the prompt carefully to find the provided algorithm. You should not be manually creating a dictionary that maps each letter to its numerical value. This is both tedious and prone to errors!

2. [25 pts] in_increasing_order

Write a function **in_increasing_order** that takes a list of strings (words) and returns True if all the strings are in increasing length order (that is, if each string is either the same length or longer than the string that came before it) and False otherwise. For example, in_increasing_order(["a", "to", "be", "what", "ready", "said", “welcome”]) would return False because, though most of the pairs of words are in increasing order, “said” is shorter than "ready".

Hint: you can use an algorithmic pattern we’ve discussed to solve this problem, but you’ll need to adjust the code slightly to make it work.

3. [25 pts] find_impact_centers

You are studying lightning strikes in an area that is prone to heavy storm activity. The area is represented in a 2D list of integers, where each value represents a section of ground. A section is assigned a value of 0 if it had no sign of heightened electrical activity during a storm, or a 1 if it did have heightened activity. You want to determine which sections were directly struck by lightning. However, heightened electrical activity in a section does not mean that lightning directly struck that location - each lightning strike causes electrical activity in the section that was struck and the sections immediately above, below, to the left, and to the right of that section.

Write a function **find_impact_centers** that takes a square 2D list of this format and returns a 2D list of the positions where lightning strikes occurred – each inner list contains the row and col of the position. For example, if the function was called on this list:

[[0, 0, 0, 0, 1],
[0, 1, 0, 1, 1],
[1, 1, 1, 0, 1],
[0, 1, 1, 0, 0],
[0, 1, 1, 1, 0]]

It would return [[1, 4], [2, 1], [4, 2]] because (1, 4), (2, 1), and (4, 2) all have values of 1 and are surrounded by other 1s. Note that (1, 4) and (4, 2) are marked as centers even though no data was collected about the value to the right of (1, 4) or the value to the bottom of (4, 2); when no data is available, you

can count it as a possible site of electrical activity. On the other hand, (3, 2) is not a center because the value to the right of the center is a 0, not a 1.

Hint: try using one of the problem-solving approaches discussed in the videos.

4. [30 pts] most_factors

Write a function **most_factors** which takes two integers and finds the integer between those two numbers (inclusive) that has the most prime factors. The function should also print a sorted list containing all the prime factors of that number. If two (or more) numbers tie for having the most prime factors, pick the number where the sum of the number's factors is largest.

Example: most_factors(100, 110) would investigate all the numbers between 100 and 110, find that 108 has the most factors (five), print [2, 2, 3, 3, 3], and return 108.

Note: when counting factors, we include factors that occur multiple times - for example, 2 divides 108 twice, so it is included twice. We can also include the number itself if the number is prime; however, we do not include 1, as 1 is not prime.

Hint: this problem is easier if you use top-down design! What helper function would be most useful to create in this problem? Perhaps something that tells you what the factors of a specific number are…

5. [50 pts] play_hangman

Hangman is a simple two-player game where one player comes up with a mystery word and the other player guesses letters in the word. If the letter is in the word, the first player writes it in all the places where it appears; if the letter is not in the word, the first player draws part of a stick figure. If the second player guesses all the letters in the word, they win; if they guess too many wrong letters and the full stick figure is drawn, they lose.

Write a function **play_hangman** which takes a string (the word to guess) and lets the user play an interactive game of Hangman. The game should implement

everything described above except drawing a stick figure. Instead, start the user at 6 incorrect guesses available and remove one for each incorrect guess.

This problem is not autograded (as you will need to take input from the user using the input function); instead, you should test it by running the function and trying it out. You will receive points for making an attempt at the problem, regardless of if it works fully or not – just make sure you press the commit button and Sync changes!

Hint: this is a complex problem. You'll want to use top-down design to make it easier!

Note: to give you an idea of what you might want to include, a sample Hangman game is provided in a comment at the bottom of the starter file.